Diseño y Análisis de Algoritmos

Ayudantía 4: Hashing y Búsqueda de Patrones (KMP)

Ayud. Ferran Rojas Andreu

Enunciado 1: Hashing — división, encadenamiento, sondeo lineal y hashing universal

Considere el conjunto de claves

$$A = [22,\ 12,\ 14,\ 5,\ 20,\ 7], \qquad n = 6,$$

que se insertan *en el orden dado* en una tabla de tamaño $m = 7$, con índices $0, \ldots, 6$. En todas las partes se usa el mismo conjunto A para poder comparar los métodos.

(A) División y encadenamiento

Use

$$h(k) = k \bmod 7$$

y resuelva las colisiones por encadenamiento, insertando siempre al inicio de la lista.

(a) Complete el arreglo de valores hash:

k	22	12	14	5	20	7
$h(k)$						

(b) Inserte todas las claves y dibuje la tabla final. Indique las cadenas y calcule α.

(c) Busque 12, 22 y 7. ¿Cuántas comparaciones hace en cada caso?

(d) Busque 13. ¿Dónde termina la búsqueda?

(e) En la cadena $T[0] = [14, 7, 0]$, elimine 14 (cabeza), 7 (medio) y 0 (cola). Explique cómo cambian los punteros.

(f) ¿Qué ocurre si se inserta 22 otra vez? ¿Y si $m = 1$?

(g) ¿Qué pasa si $n > m$? ¿Qué costo tiene una búsqueda en el peor caso?

(h) ¿Qué cambia si las listas se mantienen ordenadas?

(B) División y direccionamiento abierto (sondeo lineal)

Use la misma $h(k) = k \bmod 7$ y resuelva las colisiones por sondeo lineal:

$$\text{slot} = (h(k) + j) \bmod 7, \qquad j = 0, 1, 2, \ldots$$

Cada casilla puede estar EMPTY, ocupada o DELETED.

(a) Complete el arreglo de secuencias de sondeo:

k	22	12	14	5	20	7
sondeos						

(b) Dibuje el estado final de la tabla e indique qué clave queda en cada índice.

(c) Calcule α. ¿Por qué el sondeo lineal produce agrupamiento primario?

(d) Busque 20 y 13. ¿Cuántos sondeos necesita cada búsqueda?

(e) Elimine 22. ¿Qué valor deja en la casilla? Busque 7 después de la eliminación y explique por qué no se puede usar EMPTY.

(f) Inserte 8 después de eliminar 22. ¿Puede reutilizar la casilla? ¿Qué debe verificar antes?

(g) ¿Qué ocurre si la tabla se llena? ¿Y si α es alto o hay muchos DELETED?

(h) ¿Qué hace el algoritmo si se intenta insertar 22 otra vez?

(C) Hashing universal y encadenamiento

Use la familia

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m,$$

con

$$p = 29, \qquad m = 7, \qquad a = 3, \qquad b = 4,$$

y resuelva las colisiones por encadenamiento, insertando al inicio.

(a) Complete el arreglo:

k	22	12	14	5	20	7
$ak + b$						
$(ak + b) \bmod 29$						
$h_{a,b}(k)$						

(b) Dibuje las cadenas resultantes y calcule α.

(c) Busque 22 y 7. Elimine 5. ¿Cuántas comparaciones cuesta cada operación?

(d) Demuestre que para $x \neq y$,

$$\Pr[h_{a,b}(x) = h_{a,b}(y)] \leq \frac{1}{m}$$

cuando a, b se eligen uniformemente.

(e) ¿Qué pasa si $a = 0$? ¿Y si p no es primo? ¿Y si $p \leq \max A$?

(f) ¿Por qué elegir a, b al azar o mantenerlos secretos protege contra un adversario?

(D) Comparación

Para cada método, indique un escenario en el que sea la mejor elección y otro en el que sea una mala elección. ¿Cuándo usaría encadenamiento en vez de sondeo lineal? ¿Cuándo usaría hashing universal?

Enunciado 2: KMP y función prefijo (CLRS)

Dado un texto $T[1..n]$ y un patrón $P[1..m]$, se desea encontrar todas las ocurrencias de P en T usando el algoritmo *Knuth–Morris–Pratt* (KMP). Para ello se usa la **función prefijo** $\pi[1..m]$ (también llamada *LPS*, por "longest proper prefix which is also suffix"):

$$\pi[q] \;=\; \text{máx}\{k < q : P[1..k] \text{ es sufijo de } P[1..q]\}.$$

Las siguientes son las versiones del libro de *CLRS* :

COMPUTE-PREFIX-FUNCTION(P, m)

```
1  k ← 0
2  π[1] ← 0
3  for q = 2 to m
4      while k > 0 and P[k + 1] ≠ P[q]
5          k ← π[k]
6      if P[k + 1] = P[q]
7          k ← k + 1
8      π[q] ← k
9  return π
```

KMP-MATCHER(T, P, n, m)

```
1  π ← COMPUTE-PREFIX-FUNCTION(P, m)
2  q ← 0
3  for i = 1 to n
4      while q > 0 and P[q + 1] ≠ T[i]
5          q ← π[q]
6      if P[q + 1] = T[i]
7          q ← q + 1
8      if q = m
9          print "Pattern occurs with shift" i − m
10         q ← π[q]
```

Considere el patrón

$$P \;=\; \texttt{ababaca}$$

y el texto

$$T \;=\; \texttt{abababacaba}.$$

Determine:

(a) el valor de la función prefijo π (LPS) de P, mostrando la evolución de k y de $\pi[q]$ en cada paso del algoritmo COMPUTE-PREFIX-FUNCTION;

(b) la ejecución completa de KMP-MATCHER sobre T, mostrando el valor de q en cada posición i del texto e indicando el o los desplazamientos (*shifts*) donde ocurre P en T;

(c) el costo temporal de COMPUTE-PREFIX-FUNCTION y de KMP-MATCHER, justificando brevemente por qué el costo total es $O(n + m)$.

Enunciado 3: Depuración (debug) de LPS y KMP

Un estudiante implementó COMPUTE-PREFIX-FUNCTION y KMP-MATCHER y obtuvo los rastreos (*traces*) de la evolución de las variables que se muestran abajo. Cada rastreo contiene **exactamente dos** valores incorrectos. Su tarea es *depurarlos*.

Considere el patrón $P =$ `ababaca` y el texto $T =$ `abababacaba` del enunciado anterior, con las versiones de CLRS del pseudocódigo.

(a) Rastreo del estudiante de Compute-Prefix-Function:

q	k al inicio	evolución de k	$\pi[q]$ reportado
2	0	$P[1]{=}a \neq P[2]{=}b$	0
3	0	$P[1]{=}a = P[3]{=}a \Rightarrow k = 1$	1
4	1	$P[2]{=}b = P[4]{=}b \Rightarrow k = 2$	1
5	1	$P[2]{=}b \neq P[5]{=}a;\ k = \pi[1] = 0;\ P[1]{=}a = P[5]{=}a \Rightarrow k = 1$	1
6	1	$P[2]{=}b \neq P[6]{=}c;\ k = \pi[1] = 0;\ P[1]{=}a \neq P[6]{=}c$	0
7	0	$P[1]{=}a = P[7]{=}a \Rightarrow k = 1$	1

(b) Rastreo del estudiante de KMP-Matcher
(con la función prefijo correcta $\pi = \langle 0, 0, 1, 2, 3, 0, 1 \rangle$):

i	$T[i]$	acción	q final reportado
1	a	$P[1]{=}a = T[1] \Rightarrow q = 1$	1
2	b	$P[2]{=}b = T[2] \Rightarrow q = 2$	2
3	a	$P[3]{=}a = T[3] \Rightarrow q = 3$	3
4	b	$P[4]{=}b = T[4] \Rightarrow q = 4$	4
5	a	$P[5]{=}a = T[5] \Rightarrow q = 5$	5
6	b	$P[6]{=}c \neq T[6];\ q = \pi[5] = 3;\ P[4]{=}b = T[6] \Rightarrow q = 4$	3
7	a	$P[5]{=}a = T[7] \Rightarrow q = 5$	5
8	c	$P[6]{=}c = T[8] \Rightarrow q = 6$	6
9	a	$P[7]{=}a = T[9] \Rightarrow q = 7 = m;$ imprime shift 2; $q = \pi[7] = 1$	7
10	b	$P[2]{=}b = T[10] \Rightarrow q = 2$	2
11	a	$P[3]{=}a = T[11] \Rightarrow q = 3$	3

Determine:

(a) en el rastreo (a), cuáles son los dos valores de $\pi[q]$ incorrectos, cuál es la *causa raíz* (qué línea del pseudocódigo se aplicó mal o se omitió, y en qué iteración q), y escriba la función prefijo correcta;

(b) en el rastreo (b), cuáles son los dos valores de q incorrectos, qué línea de KMP-MATCHER se violó en cada caso, y escriba la secuencia correcta de q junto con el desplazamiento correcto;

(c) explique brevemente (2–3 líneas) por qué el error del rastreo (b) en $i = 9$ es peligroso: ¿qué pasaría en $i = 10$ si se dejara $q = 7$ sin reiniciar?

Enunciado 4: LPS del string $P\#T$

Una técnica clásica (relacionada con la función Z y con el *string matching* por prefijos) consiste en *concatenar* el patrón y el texto separados por un carácter centinela # que no aparece en P:

$$S \;=\; P \,\#\, T,$$

y computar la función prefijo π (LPS) sobre el string completo S con Compute-Prefix-Function. La idea es que $\pi[q]$ indica el largo del prefijo de P que coincide con el sufijo de $S[1..q]$; por lo tanto, cada posición q de la zona del texto con $\pi[q] = m$ (donde $m = |P|$) marca una ocurrencia de P que *termina* en esa posición de S.

Considere el patrón y el texto de los enunciados anteriores:

$$P \;=\; \texttt{ababaca}, \qquad T \;=\; \texttt{abababacaba}.$$

Determine:

(a) construya el string $S = P\#T$ y calcule la función prefijo π de *todo* S (posición por posición), mostrando la evolución de k y de $\pi[q]$. Observe que las primeras m entradas coinciden con el LPS de P del Enunciado 2;

(b) identifique todas las posiciones q de la zona del texto con $\pi[q] = m$. Para cada una, convierta el índice q de S en el índice de inicio de la ocurrencia en T y en el *shift*, y compare el resultado con el del Enunciado 2;

(c) explique por qué el carácter # es necesario: ¿qué ocurriría si se computara π sobre PT (sin el #)? Justifique además que $\pi[q] \le m$ para todo q en la zona del texto, y analice el costo temporal del método.

Sugerencia: en $S = P\#T$ el centinela # está en la posición $m+1$; el carácter $T[i]$ de la posición i del texto ocupa la posición $m + 1 + i$ de S.